Module 7 – Mernstack – React JS

Hooks (useState, useEffect, useReducer, useMemo, useRef, useCallback)

Task 1:

- Create a functional component with a counter using the useState()hook. Include buttons to increment and decrement the counter.

Ans:

```
import React, { useState } from "react";


function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h2>Counter App</h2>
      <h1>{count}</h1>

      <button onClick={increment} style={{ marginRight: "10px" }}>
        Increment
      </button>

      <button onClick={decrement}>
        Decrement
      </button>
    </div>
  );
}

export default Counter;
```

Task 2:

- Use the useEffect()hook to fetch and display data from an API when the component mounts.

Ans:
```
import React, { useState, useEffect } from "react";


function Users() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => setUsers(data))
      .catch((error) => console.error("Error fetching data:", error));
  }, []); // Runs only once when the component mounts

  return (
    <div>
      <h2>User List</h2>

      <ul>
        {users.map((user) => (
          <li key={user.id}>
            {user.name} - {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default Users;
```

Task 3:

- Create react app with use of useSelector & useDispatch.

Ans:

1. Install Redux Toolkit and React-Redux

```
npm install @reduxjs/toolkit react-redux
```

2. Create Redux Store (store.js)

```
import { configureStore, createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: {
    value: 0,
  },
```

```
  reducers: {
    increment: (state) => {
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    },
  },
});

export const { increment, decrement } = counterSlice.actions;

const store = configureStore({
  reducer: {
    counter: counterSlice.reducer,
  },
});

export default store;
```

3. Wrap App with Provider (main.jsx)

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import store from "./store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

4. Create Component Using useSelector and useDispatch (App.jsx)

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import store from "./store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Task 4:

- Create react app to avoid re-renders in react application by useRef ?

Ans:

```
import React, { useRef, useState } from "react";


function App() {
  const inputRef = useRef("");
  const [display, setDisplay] = useState("");

  console.log("Component Re-rendered");

  const handleChange = (e) => {
    inputRef.current = e.target.value;
    // No re-render occurs here
  };

  const showValue = () => {
    setDisplay(inputRef.current);
    // Re-render happens only when state changes
  };

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>useRef Example</h1>

      <input
        type="text"
        placeholder="Type something..."
        onChange={handleChange}
      />

      <br /><br />

      <button onClick={showValue}>
        Show Value
      </button>

      <h2>{display}</h2>
    </div>
  );
}

export default App;
```

Routing in React (React Router)

Task 1:

- Set up a basic React Router with two routes: one for a Home page and one for anAbout page. Display the appropriate content based on the URL.

Ans:

1. Install React Router

```
npm install react-router-dom
```

2. main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { BrowserRouter } from "react-router-dom";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

3. App.jsx

```
import React from "react";
import { Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";

function App() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link>
      </nav>

      <hr />

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </div>
  );
}
```

```
export default App;
```

Task 2:

- Create a navigation bar using React Router's Linkcomponent that allows users to switch between the Home, About, and Contact pages.

Ans:

1. Install React Router

```
npm install react-router-dom
```

2. main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

3. App.jsx

```
import React from "react";
import { Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

function App() {
  return (
    <div>
      <nav
        style={{
          padding: "15px",
          backgroundColor: "#f0f0f0",
        }}
      >
        <Link to="/" style={{ marginRight: "15px" }}>
          Home
        </Link>

        <Link to="/about" style={{ marginRight: "15px" }}>
```

```
          About
        </Link>

        <Link to="/contact">
          Contact
        </Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  );
}

export default App;
```

4. Home.jsx

```
import React from "react";

function Home() {
  return (
    <div>
      <h1>Home Page</h1>
      <p>Welcome to the Home Page.</p>
    </div>
  );
}

export default Home;
```

5. About.jsx

```
import React from "react";

function About() {
  return (
    <div>
      <h1>About Page</h1>
      <p>This is the About Page.</p>
    </div>
  );
}

export default About;
```

6. Contact.jsx

```
import React from "react";

function Contact() {
  return (
    <div>
      <h1>Contact Page</h1>
      <p>You can contact us at contact@example.com</p>
    </div>
  );
}

export default Contact;
```

React – JSON-server and Firebase Real Time Database

Task 1:

- Create a React component that fetches data from a public API (e.g., a list of users)and displays it in a table format
- Create a React app with Json-server and use Get , Post , Put , Delete & patch method on Json-server API.

Ans:

1. UsersTable.jsx

```
import React, { useEffect, useState } from "react";

function UsersTable() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((data) => setUsers(data))
      .catch((err) => console.log(err));
  }, []);

  return (
    <div>
      <h2>Users List</h2>

      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
```

```
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>

        <tbody>
          {users.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default UsersTable;
```

2: Install JSON Server

```
npm install -g json-server
```

3: Create db.json

```
{
  "users": [
    {
      "id": 1,
      "name": "Prem",
      "email": "prem@gmail.com"
    }
  ]
}
```

4: Run JSON Server

```
json-server --watch db.json --port 5000
```

5: App.jsx

```
import React, { useEffect, useState } from "react";

function App() {
```

```
const API = "http://localhost:5000/users";

const [users, setUsers] = useState([]);
const [name, setName] = useState("");

// GET
const getUsers = async () => {
  const res = await fetch(API);
  const data = await res.json();
  setUsers(data);
};

useEffect(() => {
  getUsers();
}, []);

// POST
const addUser = async () => {
  const user = {
    name: name,
    email: `${name}@gmail.com`,
  };

  await fetch(API, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(user),
  });

  getUsers();
  setName("");
};

// PUT
const updateUser = async (id) => {
  await fetch(`${API}/${id}`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      id,
      name: "Updated User",
      email: "updated@gmail.com",
    }),
  });
```

```
  getUsers();
};

// PATCH
const patchUser = async (id) => {
  await fetch(`${API}/${id}`, {
    method: "PATCH",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      name: "Patched Name",
    }),
  });

  getUsers();
};

// DELETE
const deleteUser = async (id) => {
  await fetch(`${API}/${id}`, {
    method: "DELETE",
  });

  getUsers();
};

return (
  <div style={{ padding: "20px" }}>
    <h1>JSON Server CRUD</h1>

    <input
      type="text"
      placeholder="Enter Name"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />

    <button onClick={addUser}>
      Add User
    </button>

    <hr />

    <table border="1" cellPadding="10">
      <thead>
        <tr>
```

```
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Actions</th>
          </tr>
        </thead>

        <tbody>
          {users.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>

              <td>
                <button onClick={() => updateUser(user.id)}>
                  PUT
                </button>

                <button onClick={() => patchUser(user.id)}>
                  PATCH
                </button>

                <button onClick={() => deleteUser(user.id)}>
                  DELETE
                </button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Task 2:

- Create a React app crud and Authentication with firebase API.
- Implement google Authentication with firebase API.

Ans:

1. Install Firebase

```
npm install firebase
```

2: src/firebase.js

```
import { initializeApp } from "firebase/app";


const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT.appspot.com",
  messagingSenderId: "XXXXXXXXXX",
  appId: "XXXXXXXXXXXXXX"
};

const app = initializeApp(firebaseConfig);

export default app;
```

2. CRUD with Firebase Firestore

- Install Firestore

```
npm install firebase
```

- Update firebase.js

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
};

const app = initializeApp(firebaseConfig);

export const db = getFirestore(app);

export default app;
```

- Crud.jsx

```
import React, { useState, useEffect } from "react";
import {
  collection,
  addDoc,
  getDocs,
  updateDoc,
```

```
  deleteDoc,
  doc,
} from "firebase/firestore";

import { db } from "./firebase";

function Crud() {
  const [name, setName] = useState("");
  const [users, setUsers] = useState([]);

  const usersCollection = collection(db, "users");

  // GET
  const getUsers = async () => {
    const data = await getDocs(usersCollection);

    setUsers(
      data.docs.map((doc) => ({
        ...doc.data(),
        id: doc.id,
      }))
    );
  };

  useEffect(() => {
    getUsers();
  }, []);

  // POST
  const addUser = async () => {
    await addDoc(usersCollection, {
      name,
    });

    setName("");
    getUsers();
  };

  // UPDATE
  const updateUser = async (id) => {
    const userDoc = doc(db, "users", id);

    await updateDoc(userDoc, {
      name: "Updated User",
    });

    getUsers();
  };
```

```
  // DELETE
  const deleteUser = async (id) => {
    const userDoc = doc(db, "users", id);

    await deleteDoc(userDoc);

    getUsers();
  };

  return (
    <div>
      <h1>Firebase CRUD</h1>

      <input
        type="text"
        placeholder="Enter Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <button onClick={addUser}>Add User</button>

      <hr />

      {users.map((user) => (
        <div key={user.id}>
          <h3>{user.name}</h3>

          <button onClick={() => updateUser(user.id)}>
            Update
          </button>

          <button onClick={() => deleteUser(user.id)}>
            Delete
          </button>
        </div>
      ))}
    </div>
  );
}

export default Crud;
```

3. Firebase Email/Password Authentication

- Update firebase.js

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseapp.com",
};

const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);

export default app;
```

- Auth.jsx

```
import React, { useState } from "react";

import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
} from "firebase/auth";

import { auth } from "./firebase";

function Auth() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const register = async () => {
    await createUserWithEmailAndPassword(
      auth,
      email,
      password
    );

    alert("User Registered");
  };

  const login = async () => {
    await signInWithEmailAndPassword(
      auth,
      email,
      password
    );

    alert("Login Successful");
  };
```

```
  return (
    <div>
      <h1>Firebase Authentication</h1>

      <input
        type="email"
        placeholder="Email"
        onChange={(e) => setEmail(e.target.value)}
      />

      <br /><br />

      <input
        type="password"
        placeholder="Password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <br /><br />

      <button onClick={register}>
        Register
      </button>

      <button onClick={login}>
        Login
      </button>
    </div>
  );
}

export default Auth;
```

4. Google Authentication with Firebase

- Update firebase.js

```
import { initializeApp } from "firebase/app";
import {
  getAuth,
  GoogleAuthProvider,
} from "firebase/auth";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseapp.com",
};
```

```
const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);

export const googleProvider =
  new GoogleAuthProvider();

export default app;
```

- GoogleLogin.jsx

```
import React from "react";

import {
  signInWithPopup,
  signOut,
} from "firebase/auth";

import {
  auth,
  googleProvider,
} from "./firebase";

function GoogleLogin() {
  const googleLogin = async () => {
    try {
      const result = await signInWithPopup(
        auth,
        googleProvider
      );

      console.log(result.user);

      alert(
        `Welcome ${result.user.displayName}`
      );
    } catch (error) {
      console.log(error);
    }
  };

  const logout = async () => {
    await signOut(auth);
    alert("Logged Out");
  };

  return (
    <div>
```

```
      <h1>Google Authentication</h1>

      <button onClick={googleLogin}>
        Sign In with Google
      </button>

      <button onClick={logout}>
        Logout
      </button>
    </div>
  );
}

export default GoogleLogin;
```

Task 3:

- Implement error handling and loading states for the API call. Display a loadingspinner while the data is being fetched.

Ans:

- Users.jsx

```
import React, { useEffect, useState } from "react";

function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const fetchUsers = async () => {
      try {
        setLoading(true);

        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users"
        );

        if (!response.ok) {
          throw new Error("Failed to fetch data");
        }

        const data = await response.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
```

```
      setLoading(false);
    }
  };

  fetchUsers();
}, []);

// Loading State
if (loading) {
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <div className="spinner"></div>
      <h3>Loading...</h3>
    </div>
  );
}

// Error State
if (error) {
  return (
    <div style={{ color: "red", textAlign: "center" }}>
      <h3>Error: {error}</h3>
    </div>
  );
}

return (
  <div>
    <h2>User List</h2>

    <table border="1" cellPadding="10">
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
        </tr>
      </thead>

      <tbody>
        {users.map((user) => (
          <tr key={user.id}>
            <td>{user.id}</td>
            <td>{user.name}</td>
            <td>{user.email}</td>
          </tr>
        ))}
      </tbody>
```

```
      </table>
    </div>
  );
}

export default Users;
```

- App.css

```
.spinner {
  width: 50px;
  height: 50px;
  border: 5px solid #ddd;
  border-top: 5px solid blue;
  border-radius: 50%;
  animation: spin 1s linear infinite;
  margin: auto;
}

@keyframes spin {
  100% {
    transform: rotate(360deg);
  }
}
```

Context API

Task 1:

- Create a simple theme toggle (light/dark mode) using the Context API. The themestate should be shared across multiple components.

Ans:

1: ThemeContext.jsx

```
import React, { createContext, useState } from "react";

export const ThemeContext = createContext();

export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState("light");

  const toggleTheme = () => {
    setTheme(theme === "light" ? "dark" : "light");
  };

  return (
```

```
    <ThemeContext.Provider
      value={{ theme, toggleTheme }}
    >
      {children}
    </ThemeContext.Provider>
  );
};
```

2: main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { ThemeProvider } from "./ThemeContext";

ReactDOM.createRoot(document.getElementById("root")).render(
  <ThemeProvider>
    <App />
  </ThemeProvider>
);
```

3: Navbar.jsx

```
import React, { useContext } from "react";
import { ThemeContext } from "./ThemeContext";

function Navbar() {
  const { theme } = useContext(ThemeContext);

  return (
    <nav
      style={{
        padding: "20px",
        background:
          theme === "light" ? "#f5f5f5" : "#222",
        color:
          theme === "light" ? "#000" : "#fff",
      }}
    >
      <h2>Navbar Component</h2>
    </nav>
  );
}

export default Navbar;
```

4: Content.jsx

```
import React, { useContext } from "react";
import { ThemeContext } from "./ThemeContext";

function Content() {
  const { theme } = useContext(ThemeContext);

  return (
    <div
      style={{
        padding: "20px",
        background:
          theme === "light" ? "#fff" : "#333",
        color:
          theme === "light" ? "#000" : "#fff",
      }}
    >
      <h2>Content Component</h2>
      <p>
        Current Theme: {theme}
      </p>
    </div>
  );
}

export default Content;
```

5: App.jsx

```
import React, { useContext } from "react";
import { ThemeContext } from "./ThemeContext";
import Navbar from "./Navbar";
import Content from "./Content";

function App() {
  const { theme, toggleTheme } =
    useContext(ThemeContext);

  return (
    <div
      style={{
        minHeight: "100vh",
        background:
          theme === "light" ? "#ffffff" : "#121212",
        color:
          theme === "light" ? "#000000" : "#ffffff",
      }}
    >
      <button
```

```
        onClick={toggleTheme}
        style={{
          margin: "20px",
          padding: "10px 20px",
        }}
      >
        Toggle Theme
      </button>

      <Navbar />
      <Content />
    </div>
  );
}

export default App;
```

Task 2:

- Use the Context API to create a global user authentication system. If the user islogged in, display a welcome message; otherwise, prompt them to log in.

Ans:

1: AuthContext.jsx

```
import React, { createContext, useState } from "react";

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);

  const login = () => {
    setUser({
      name: "Prem",
    });
  };

  const logout = () => {
    setUser(null);
  };

  return (
    <AuthContext.Provider
      value={{ user, login, logout }}
    >
      {children}
```

```
    </AuthContext.Provider>
  );
};
```

2: main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { AuthProvider } from "./AuthContext";

ReactDOM.createRoot(document.getElementById("root")).render(
  <AuthProvider>
    <App />
  </AuthProvider>
);
```

3: Navbar.jsx

```
import React, { useContext } from "react";
import { AuthContext } from "./AuthContext";

function Navbar() {
  const { user } = useContext(AuthContext);

  return (
    <nav
      style={{
        padding: "15px",
        background: "#f1f1f1",
      }}
    >
      <h2>Authentication App</h2>

      {user ? (
        <p>Welcome, {user.name}</p>
      ) : (
        <p>Please Login</p>
      )}
    </nav>
  );
}

export default Navbar;
```

4: Login.jsx

```jsx
import React, { useContext } from "react";
import { AuthContext } from "./AuthContext";

function Login() {
  const { user, login, logout } =
    useContext(AuthContext);

  return (
    <div style={{ padding: "20px" }}>
      {user ? (
        <>
          <h3>Welcome {user.name}</h3>

          <button onClick={logout}>
            Logout
          </button>
        </>
      ) : (
        <>
          <h3>You are not logged in</h3>

          <button onClick={login}>
            Login
          </button>
        </>
      )}
    </div>
  );
}

export default Login;
```

5: App.jsx

```jsx
import React from "react";
import Navbar from "./Navbar";
import Login from "./Login";

function App() {
  return (
    <div>
      <Navbar />
      <Login />
    </div>
  );
}

export default App;
```

State Management (Redux, Redux-Toolkit or Recoil)

Task 1:

- Create a simple counter application using Redux for state management. Implementactions to increment and decrement the counter.

Ans:

1: Install Redux Toolkit and React-Redux

```
npm install @reduxjs/toolkit react-redux
```

2: store.js

```
import { configureStore, createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: {
    count: 0,
  },
  reducers: {
    increment: (state) => {
      state.count += 1;
    },
    decrement: (state) => {
      state.count -= 1;
    },
  },
});

export const { increment, decrement } = counterSlice.actions;

const store = configureStore({
  reducer: {
    counter: counterSlice.reducer,
  },
});

export default store;
```

3: main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import store from "./store";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

4: App.jsx

```
import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement } from "./store";

function App() {
  const count = useSelector(
    (state) => state.counter.count
  );

  const dispatch = useDispatch();

  return (
    <div
      style={{
        textAlign: "center",
        marginTop: "50px",
      }}
    >
      <h1>Redux Counter App</h1>

      <h2>Count: {count}</h2>

      <button
        onClick={() => dispatch(increment())}
        style={{
          marginRight: "10px",
          padding: "10px 20px",
        }}
      >
        Increment
      </button>

      <button
        onClick={() => dispatch(decrement())}
        style={{
          padding: "10px 20px",
        }}
      >
        Decrement
```

```
      </button>
    </div>
  );
}

export default App;
```

Task 2:

- Build a todo list application using Recoil for state management. Allow users to add,remove, and mark tasks as complete.

Ans:

1: Install Recoil

```
npm install recoil
```

2: atoms/todoAtom.js

```
import { atom } from "recoil";

export const todoState = atom({
  key: "todoState",
  default: [],
});
```

3: main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { RecoilRoot } from "recoil";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <RecoilRoot>
    <App />
  </RecoilRoot>
);
```

4: App.jsx

```
import React, { useState } from "react";
import { useRecoilState } from "recoil";
import { todoState } from "./atoms/todoAtom";
```

```
function App() {
  const [todos, setTodos] = useRecoilState(todoState);
  const [task, setTask] = useState("");

  // Add Task
  const addTodo = () => {
    if (!task.trim()) return;

    const newTodo = {
      id: Date.now(),
      text: task,
      completed: false,
    };

    setTodos([...todos, newTodo]);
    setTask("");
  };

  // Remove Task
  const removeTodo = (id) => {
    setTodos(todos.filter((todo) => todo.id !== id));
  };

  // Toggle Complete
  const toggleTodo = (id) => {
    setTodos(
      todos.map((todo) =>
        todo.id === id
          ? { ...todo, completed: !todo.completed }
          : todo
      )
    );
  };

  return (
    <div
      style={{
        width: "400px",
        margin: "50px auto",
      }}
    >
      <h1>Recoil Todo App</h1>

      <input
        type="text"
        placeholder="Enter Task"
        value={task}
        onChange={(e) => setTask(e.target.value)}
```

```
      />

      <button onClick={addTodo}>
        Add
      </button>

      <ul>
        {todos.map((todo) => (
          <li
            key={todo.id}
            style={{
              margin: "10px 0",
            }}
          >
            <span
              style={{
                textDecoration: todo.completed
                  ? "line-through"
                  : "none",
                marginRight: "10px",
              }}
            >
              {todo.text}
            </span>

            <button
              onClick={() => toggleTodo(todo.id)}
            >
              {todo.completed
                ? "Undo"
                : "Complete"}
            </button>

            <button
              onClick={() => removeTodo(todo.id)}
              style={{ marginLeft: "10px" }}
            >
              Delete
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Task 3:

- Build a crud application using Redux-Toolkit for state management. Allow users to add,remove, delete and update.

Ans:

1: Install Redux Toolkit

```
npm install @reduxjs/toolkit react-redux
```

2: features/userSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  users: [],
};

const userSlice = createSlice({
  name: "users",
  initialState,
  reducers: {
    addUser: (state, action) => {
      state.users.push(action.payload);
    },

    deleteUser: (state, action) => {
      state.users = state.users.filter(
        (user) => user.id !== action.payload
      );
    },

    updateUser: (state, action) => {
      const { id, name } = action.payload;

      const user = state.users.find(
        (user) => user.id === id
      );

      if (user) {
        user.name = name;
      }
    },
  },
});

export const {
  addUser,
```

```
  deleteUser,
  updateUser,
} = userSlice.actions;

export default userSlice.reducer;
```

3: store.js

```
import { configureStore } from "@reduxjs/toolkit";
import userReducer from "./features/userSlice";

export const store = configureStore({
  reducer: {
    users: userReducer,
  },
});
```

4: main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import { store } from "./store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

5: App.jsx

```
import React, { useState } from "react";
import {
  useSelector,
  useDispatch,
} from "react-redux";

import {
  addUser,
  deleteUser,
  updateUser,
} from "./features/userSlice";

function App() {
  const [name, setName] = useState("");
```

```
const [editId, setEditId] = useState(null);

const users = useSelector(
  (state) => state.users.users
);

const dispatch = useDispatch();

const handleSubmit = () => {
  if (!name.trim()) return;

  if (editId) {
    dispatch(
      updateUser({
        id: editId,
        name,
      })
    );

    setEditId(null);
  } else {
    dispatch(
      addUser({
        id: Date.now(),
        name,
      })
    );
  }

  setName("");
};

const handleEdit = (user) => {
  setName(user.name);
  setEditId(user.id);
};

return (
  <div
    style={{
      width: "500px",
      margin: "50px auto",
    }}
  >
    <h1>Redux Toolkit CRUD App</h1>

    <input
      type="text"
```

```
        placeholder="Enter Name"
        value={name}
        onChange={(e) =>
          setName(e.target.value)
        }
      />

      <button onClick={handleSubmit}>
        {editId ? "Update" : "Add"}
      </button>

      <hr />

      {users.map((user) => (
        <div
          key={user.id}
          style={{
            display: "flex",
            gap: "10px",
            marginBottom: "10px",
          }}
        >
          <span>{user.name}</span>

          <button
            onClick={() => handleEdit(user)}
          >
            Edit
          </button>

          <button
            onClick={() =>
              dispatch(deleteUser(user.id))
            }
          >
            Delete
          </button>
        </div>
      ))}
    </div>
  );
}

export default App;
```